electronics

MDPI

Article

A Dual-Branch Structure Network of Custom Computing for Multivariate Time Series

Jingfeng Yu [1,2], Yingqi Feng [2,3] and Zunkai Huang [2,3,*]

[1] School of Information Science and Technology, Shanghaitech University, Shanghai 201210, China; yujf@shanghaitech.edu.cn
[2] Shanghai Advanced Research Institute, Chinese Academy of Sciences, Shanghai 201210, China; fengyq@sari.ac.cn
[3] University of Chinese Academy of Sciences, Beijing 100049, China
* Correspondence: huangzk@sari.ac.cn

Abstract: Time series are a common form of data, which are of great importance in multiple fields. Multivariate time series whose relationship of dimension is indeterminacy are particularly common within these. For multivariate time series, we proposed a dual-branch structure model, composed of an attention branch and a convolution branch, respectively. The algorithm proposed in our work is implemented for custom computing optimization and deployed on the Xilinx Ultra 96V2 device. Comparative results with other state-of-the-art time series algorithms on public datasets indicate that the proposed method achieves optimal performance.The power consumption of the system is 6.38 W, which is 47.02 times lower than that of a GPU.

Keywords: time series; custom computing; convolution; attention mechanism

check for updates

Citation: Yu, J.; Feng, Y.; Huang, Z. A Dual-Branch Structure Network of Custom Computing for Multivariate Time Series. *Electronics* **2024**, *13*, 1357. https://doi.org/10.3390/electronics13071357

Academic Editor: Maciej Ławryńczuk

Received: 29 February 2024
Revised: 22 March 2024
Accepted: 2 April 2024
Published: 3 April 2024

Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

Time series are a common form of data, which are of great importance in multiple fields, such as power systems, medical study, finance analysis, and so on. Multivariate time series whose relationship of dimension is indeterminacy are particularly common within these.

In 2017, the Google team improved Transformer-like models [1], abandoning traditional architectures such as recurrent neural networks and convolutional neural networks. They constructed models by stacking encoders and decoders, with each encoder and decoder having the same structure. In 2018, Google introduced the BERT model, which only utilizes the encoding part of Transformer. It is a bidirectional network capable of simultaneously observing contextual information from both directions [2]. In 2019, OpenAI proposed the GPT model, which utilizes the decoding part of Transformer. It is a unidirectional network with causality in its processing [3]. In 2020, George Zerveas from Brown University and IBM jointly released the TST model [4]. They utilized the Transformer model for analyzing multidimensional time series, demonstrating impressive performance in prediction and classification tasks. Unlike the original BERT model applied to NLP problems, TST employs BatchNorm instead of LayerNorm. Proposed by Ismail Fawaz et al. in 2020, InceptionTime adapts the inception module to the unique characteristics of time series data [5]. It introduces parallel pathways with different filter sizes to effectively capture diverse temporal patterns and hierarchical features. This model has proven to be particularly adept at handling various time series analysis tasks, including classification and forecasting. Its performance across different benchmarks highlights its effectiveness in capturing intricate patterns within time series datasets. Proposed by Y. Lin, I. Koprinska, and M. Rana in 2021 [6], the Temporal Convolutional Attention Neural Network focuses on advancing the field of time series forecasting. This model uniquely combines the efficiency of Convolutional Neural Networks (CNNs) in processing temporal data with the

capabilities of Attention Mechanisms to capture long-term dependencies more effectively in time series.

In recent years, significant strides have been made in multivariate time series forecasting and classification. Shih et al. introduced an innovative method to capture temporal patterns in time series data [7], emphasizing multivariate forecasting. This method demonstrates the growing importance of nuanced pattern recognition in time series analysis. Qin et al. proposed a dual-stage attention mechanism [8]. This method significantly enhances prediction accuracy in complex, multivariate time series contexts, highlighting the efficacy of sophisticated neural network architectures in handling intricate data patterns. Khodabakhsh et al. focused on the application of LSTM networks for multivariate time series forecasting [9]. Their paper addresses the efficiency and effectiveness of batch-processing techniques, underscoring the importance of computational efficiency in handling large-scale time series data.

MengHao Guo et al. proposed an external attention mechanism, which utilizes two external learnable and shareable memory modules consisting of two linear layers and two normalization layers, reducing the algorithm complexity to linear [10]. Similarly, Chuhan Wu designed additive attention based on classic attention structures and proposed Fastformer [11]. This attention mechanism models global information and further processes elements based on their relationships with the global context, rather than computing pairwise relationships between elements. In this manner, Fastformer achieves linear complexity as well.

Layer normalization (LayerNorm) and residual connection structures can stabilize deep neural networks during training, avoiding unstable gradient changes and model degradation. However, LayerNorm operations have relatively high computational complexity, leading some researchers to analyze and improve these operations. Xu et al. pointed out through empirical observations and experimental analysis that the derivatives of data mean and variance are crucial factors for the excellent performance of LayerNorm, while other learnable parameters have minimal impact on experimental results [12]. Shen et al. discussed the differences between BatchNorm and LayerNorm and proposed PowerNorm based on BatchNorm [13]. PowerNorm no longer requires normalization operations with a mean of 0, and it utilizes the square mean of sequences instead of variance.

There has been significant optimization work on feedforward layers as well. The GPT model proposed by OpenAI replaced the ReLU activation function in the feedforward layers with GELU, which has almost become a standard setting in language modeling [3]. Meanwhile, Shazeer et al. utilized GLU instead of ReLU, which is an activation function similar to the gating structure in LSTM [14]. Apart from activation functions, some researchers aimed to alter the simple structure of feedforward layers to increase model capacity. The Mixture-of-Experts (MoE) with auxiliary learnable parameters is a representative work in this regard [15]. Conversely, many researchers have sought to streamline the structure of feedforward layers. Zhanghao Wu from MIT's Han Song Laboratory pointed out that in language models, feedforward layers themselves do not effectively capture contextual information but consume a significant amount of computational resources [16]. He improved the traditional bottleneck-style feedforward layer structure, keeping the channel dimension unchanged while reducing computation and parameters. Sukhbaatar et al. suggested adding a matrix with learnable parameters to the attention layer to mimic the role of the feedforward layer in the model, thereby discarding the feedforward layer and simplifying the structure [17]. Yang et al. directly discarded the feedforward layer in the decoding part of the Transformer model. From experimental data, they concluded that although feedforward layers have a large volume of data, their computational efficiency is low. Removing this part directly does not result in significant losses; instead, it greatly improves training and inference speed [18].

The large number of parameters in fully connected layers can slow down training speed and potentially lead to overfitting. Min Lin first proposed simplifying fully connected layers using global average pooling to reduce parameter count and computational

complexity while enhancing experimental results [19]. Due to the simplicity and efficiency of global average pooling, it has been widely adopted in convolutional neural networks. Scholars from Zhejiang University found a linear relationship between the fundamental component of 2D discrete cosine transform and global average pooling. Consequently, they introduced frequency domain features into pooling operations for the first time, proposing FcaNet [20]. Proposed by Lee-Thorp et al. in 2020, FNet deviates from the conventional Transformer architecture by replacing the self-attention mechanism with a simple and efficient Fourier transform [21]. The key idea is to leverage the Fourier transform's ability to capture global dependencies in a sequence, offering an alternative approach to sequence modeling. The FNet model has shown promising results in various natural language processing and time series tasks, demonstrating competitive performance while mitigating some of the computational complexities associated with traditional self-attention mechanisms. This unique design choice has sparked interest in exploring alternative architectures for sequence modeling.

We address algorithm design and hardware deployment for multivariate time series, focusing on the integration of both domains. Our proposed method, characterized by its efficiency and high parallelism, optimizes computational processes within hardware architectures, enhancing analysis speed. In response to the complex traits of multivariate time series, such as unpredictable correlations among dimensions, subtle features, and varied historical data needs, we employ a unified approach, utilizing a deep learning network algorithm tailored for this context. Our methodology, validated across diverse multidimensional datasets, exhibits proficient feature extraction and strong generalization. Emphasizing custom computing compatibility, our design achieves substantial parallelism, exploiting custom computing's parallel processing benefits. Moreover, it enables low-power, accurate processing for a spectrum of time series tasks, demonstrating the effectiveness of our integrated approach in algorithm and hardware synergy.

2. Methodology

2.1. Model Architecture

The model is divided into two parts: the attention branch and the convolution branch. Firstly, the original data x are mapped to x_{conv} using a linear layer, where the channel dimensions of x and x_{conv} are m and $6n$, respectively, with $m \ll 6n$. x_{conv} is directly transmitted to the convolutional branch. For the attention branch, we additionally introduce learnable positional encoding information L, and the input to the attention branch is $x_{\text{att}} = x_{\text{conv}} + L$. Positional encoding is an important encoding method for Transformer-like models because the self-attention mechanism alone cannot distinguish the sequential order of elements. However, in fields such as text modeling and speech recognition, the order relationship of elements is crucial. Therefore, additional positional encoding is particularly important for the attention mechanism. A common approach is to encode the position information into vectors, which are then input into the model. The convolutional module does not require positional encoding since convolution analyzes sequences in order during computation.

2.2. Attention Branch

The attention branch is constructed by several submodules, each comprising a self-attention layer and a feedforward layer. This work draws inspiration from Google's FNet and utilizes a two-dimensional Discrete Fourier Transform (DFT) to model the self-attention layer. Implementing Fourier transforms in time series analysis presents a blend of advantages. Their main strengths include a significant boost in processing speed, which is especially beneficial for handling long sequences in time series data. This method simplifies the model architecture while effectively handling tasks like anomaly detection and trend analysis in time series data. Fourier transforms also have a lower memory footprint, which is advantageous in applications where model size and computational resources are limiting factors. However, there are notable drawbacks. The accuracy of Fourier transform-based

methods may not always match the precision of more conventional time series analysis techniques. Additionally, their computational efficiency can be dependent on the specific hardware used and the length of the time series. The definition of the one-dimensional DFT is as follows: Given a sequence x_n with a length of H, the H-point DFT of x_n is defined as:

$$X(k) = \mathrm{DFT}[x(h)] = \sum_{h=0}^{H-1} x(h) e^{-\frac{2\pi i}{H} hk}, \quad 0 \leq k \leq H-1 \tag{1}$$

The computational complexity of the DFT is $O(N^2)$. We utilize fast Fourier transform (FFT) to accelerate DFT. FFT is the most commonly used method to implement DFT, leveraging the symmetry and periodicity of DFT to reduce the computational complexity to $O(N \log N)$. The two-dimensional FFT in the attention branch operates separately on the channel and sequence length dimensions, ultimately outputting only the real part of the data. In the design of the feedforward layer in the classical Transformer model, the feedforward layer consists of two linear layers. Each linear layer requires an "Add + Norm" operation, meaning the data are added to the upper-layer data and then normalized. In the attention branch design of this paper, only the "Add + Norm" operation is retained, and linear layers are not added. LayerNorm is a common normalization method in NLP tasks, but this paper opts for BatchNorm. In NLP problems, data mapped from words to sentences often exhibit large fluctuations, whereas time series data tend to have smaller fluctuations. BatchNorm can mitigate the impact of outliers in the data, making it more suitable for time series rather than NLP problems. The structural design of the feedforward layer will be discussed in detail in the experimental section later in the paper. In summary, the formula for the attention module is as follows:

$$y = \mathrm{BatchNorm}(x + \Re(F_{\mathrm{seq}}(F_{\mathrm{channel}}(x)))) \tag{2}$$

Here, x and y represent the module's input and output, respectively. F_{channel} and F_{seq} denote the fast Fourier transforms in the channel and sequence length dimensions, and $\Re$ represents the real part.

2.3. Convolution Branch

Compared to the self-attention layer, the convolutional branch focuses more on processing partial information. Drawing inspiration from the InceptionTime model, this paper adopts a design with several parallel convolutional layers. However, unlike InceptionTime, we discard both bottleneck and residual structures, as these structures have minimal impact on experimental results. The convolutional branch is comprised of six stacked convolutional modules. Each module consists of six parallel one-dimensional convolutional layers with different kernel sizes but identical input and output dimensions. The default kernel sizes are {1, 3, 5, 11, 25, 49}, allowing the extraction of features at different scales. The input and output dimensions are 6n and n, respectively, enabling feature extraction and dimensionality reduction concurrently, achieving a bottleneck-like effect. Although the convolutional layer with a kernel size of 1 cannot capture features at specific time scales, it effectively preserves the information of the original sequence, resembling the effect of a residual structure. The convolutional branch in this paper simplifies the max-pooling layer in the InceptionTime model, enabling parallel computation of the six convolutional layers to enhance parallelism. The convolutional results are concatenated along the channel dimension, restoring the module's output dimension to 6n for input into the next convolutional branch.

2.4. Global Frequency Pooling

After the parallel computation of the attention and convolution branches, they are concatenated along the channel dimension and input into a global frequency pooling layer, where the channel dimension is set to $12n$. Following the design of the classical Transformer structure, several fully connected layers are expected to be added for the final output. This

paper introduces frequency domain information into the pooling operation and proposes an innovative global frequency pooling layer. This layer extracts features from the results of the dual-branch computation, thereby streamlining the model structure.

The one-dimensional Discrete Cosine Transform (DCT) is defined as follows:

$$\begin{bmatrix} C(0) \\ C(1) \\ \vdots \\ C(n-1) \end{bmatrix} = \frac{1}{\sqrt{N}} \begin{bmatrix} 1 & 1 & \cdots & 1 \\ \sqrt{2}\cos\left(\frac{\pi}{2N}\right) & \sqrt{2}\cos\left(\frac{3\pi}{2N}\right) & \cdots & \sqrt{2}\cos\left(\frac{(2N-1)\pi}{2N}\right) \\ \vdots & \vdots & \ddots & \vdots \\ \sqrt{2}\cos\left(\frac{(N-1)\pi}{2N}\right) & \sqrt{2}\cos\left(\frac{3(N-1)\pi}{2N}\right) & \cdots & \sqrt{2}\cos\left(\frac{(2N-1)(N-1)\pi}{2N}\right) \end{bmatrix} \begin{bmatrix} x(0) \\ x(1) \\ \vdots \\ x(N-1) \end{bmatrix} \tag{3}$$

Here, C represents the sequence obtained after the DCT transformation, and x is the original sequence.

As observed from the previous DCT and DFT transformation formulas, the first element of the output results from both DCT and DFT, i.e., the "DC component", involves summing all the data in the sequence. This process is analogous to global average pooling. The difference lies in the normalization coefficients of DFT and DCT, which do not affect the results. Pooling operations are employed to extract effective features and reduce dimensions. Global average pooling characterizes only the "DC component" information of the sequence. To obtain more comprehensive features, this paper introduces other frequency domain information, namely the "AC components", enriching the feature extraction methods from a frequency domain perspective.

The process of global frequency pooling is as follows: Firstly, the data are transformed into the frequency domain to acquire frequency domain signals. Subsequently, P features (defaulting to 5) are selected as the output. Four methods for selecting frequency domain features are designed in this paper: The first method involves selecting several low-frequency components from low to high. The second method is to select several maximum values from the frequency domain signal. The third and fourth methods involve uniformly dividing the frequency domain into several segments and selecting the average or maximum value for each frequency domain band. The last two methods can more comprehensively reflect information from the entire frequency domain.

The frequency pooling layer analyzes each dimension separately. If the input data have dimensions of $(12n, seq_len)$, the output data dimensions after the frequency pooling layer will be $(12n, P)$, where $P \ll seq_len$. The specific experimental section will discuss in detail the impact of different frequency domain processing methods on the experimental results.

3. Experiment Results and Evaluation

3.1. Experiment Setup

We conducted experiments in various aspects, including position encoding, the attention mechanism, the feedforward layer, and the global frequency pooling layer. Comparisons are made with other state-of-the-art methods in time series processing. The network model is implemented using Python 3.6.8 + PyTorch 1.10.0, and the experiments are conducted on a Tesla V100-SXM2-32GB GPU. The experimental data are sourced from multidimensional datasets in publicly available datasets such as UCR, UEA, and Monash [22], categorized into classification and prediction tasks. The dataset configurations, including training set size, sequence length, and sequence dimensions, remain consistent with the original datasets.

Standardization normalization is applied to all input data in each dimension, following the formula:

$$x' = \frac{x - \mu}{\sigma} \tag{4}$$

where x' is the standardized normalized data, x is the original data, μ is the mean of the original data, and σ is the standard deviation of the original data. Each experimental data point is obtained by averaging the results of 10 experiments. Each experiment is run for a sufficient amount of time to ensure model convergence.

3.2. Position Encoding

To begin with, we investigate the impact of position encoding on the experimental results. In this segment of the experiments, the global frequency pooling layer incorporates discrete cosine transform for low-dimensional feature extraction. The classification results are presented in Table 1, with accuracy as the performance metric, while the prediction task outcomes are displayed in Table 2, evaluated based on the Root Mean Square Error (RMSE). Consistent with these metrics, other experiments are assessed.

Table 1. Compare the accuracy of non-position encoding and position encoding in classification tasks.

Dataset	Non-Position Encoding	Position Encoding
EthanolConcentration	0.2638	0.2586
FaceDetection	0.6700	0.6711
Handwriting	0.6624	0.6647
Heartbeat	0.6488	0.6829
JapaneseVowels	0.8784	0.9162
NATOPS	0.8667	0.8833
PEMS-SF	0.8382	0.8497
PhonemeSpectra	0.2216	0.2320
RacketSports	0.8684	0.8553
SpokenArabicDigits	0.7435	0.7835
StandWalkJump	0.4000	0.4667
UWaveGestureLibrary	0.8906	0.8844

Tasks underlined exhibit a substantial improvement.

Table 2. Compare the RMSE of non-position encoding and position encoding in forecast tasks.

Dataset	Non-Position Encoding	Position Encoding
AppliancesEnergy	3.1002	3.0068
Covid3Month	0.1172	0.0809
FloodModeling1	0.0328	0.0240
FloodModeling2	0.0240	0.0209
FloodModeling3	0.0495	0.0413
IEEEPPG	30.0839	25.1032
LiveFuelMoistureContent	51.5009	51.3536

The experimental results from Tables 1 and 2 underscore the pivotal significance of position encoding. Position encoding demonstrates the ability to enhance model performance to a certain extent. Particularly notable is the reduction in RMSE across seven distinct prediction tasks. Among the twelve classification tasks, seven tasks (indicated by underlined entries in the tables) exhibit a substantial improvement in classification accuracy through the incorporation of position encoding. This highlights the crucial role of position encoding information for both the attention branch and the overall model, despite the presence of a convolutional branch capable of sequentially processing data.

It is noteworthy that, when facing diverse time series tasks, position encoding is not a panacea. For some tasks, the performance improvement brought by position encoding is modest, and in isolated cases, it even leads to a certain degree of degradation in experimental results. However, overall, the addition of position encoding proves effective in enhancing the model's overall performance.

3.3. Attention Mechanism

The experimental design involves a specific analysis of the comparative performance between the classical self-attention mechanism and the two-dimensional fast Fourier transform. Tables 3 and 4 provide a detailed juxtaposition of their respective experimental results in classification and prediction tasks.

Table 3. Comparison of self-attention mechanism and 2DFFT in classification tasks.

Dataset	Classical Self-Attention		2DFFT	
	Accuracy	Times (s)	Accuracy	Times (s)
EthanolConcentration	0.2395	46	0.2586	31
FaceDetection	0.6802	638	0.6595	507
Handwriting	0.6412	29	0.6647	22
Heartbeat	0.6683	23	0.6829	17
JapaneseVowels	0.9162	30	0.9162	22
NATOPS	0.8722	17	0.8833	12
PEMS-SF	0.8439	33	0.8035	23
PhonemeSpectra	0.1915	390	0.2320	283
RacketSports	0.8750	18	0.8553	12
SpokenArabicDigits	0.8040	688	0.8835	430
StandWalkJump	0.3333	8	0.4667	6
UWaveGestureLibrary	0.8594	13	0.8844	10

Table 4. Comparison of self-attention mechanism and 2DFFT in forecast tasks.

Dataset	Classical Self-Attention		2DFFT	
	RMSE	Times (s)	RMSE	Times (s)
AppliancesEnergy	2.9991	23	3.0068	6
Covid3Month	0.0550	17	0.0809	10
FloodModeling1	0.0416	51	0.0240	32
FloodModeling2	0.0191	52	0.0209	31
FloodModeling3	0.0258	44	0.0413	29
IEEEPPG	29.6591	198	25.1032	143
LiveFuelMoistureContent	51.6052	396	51.3536	262

Comparison of Experimental Results: The 2DFFT model demonstrates excellent performance in effectively approximating the classical self-attention mechanism. Remarkably, in the context of 12 classification tasks, 2DFFT outperforms the classical self-attention mechanism in 6 tasks (underscored data in the table). In the case of 7 prediction tasks, 2DFFT surpasses the classical self-attention mechanism in 2 tasks. Although 2DFFT leads to a slight reduction in experimental results in some tasks, the decline is not significant. Thus, the conclusion can be drawn that, in the realm of time series tasks, 2DFFT serves as an effective surrogate for the classical self-attention mechanism.

Subsequently, an analysis of the parameters and computational requirements of the two methods is conducted. As observed from the table, 2DFFT exhibits a faster computational speed compared to the classical self-attention mechanism, resulting in reduced training times across all tasks. Assume the number of channels is d and the sequence length is S. The classical self-attention mechanism requires a number of operations (i.e., multiply-add operations) of $2dS^2 + 4d^2S$, whereas the operation count for the 2DFFT transformation is $dS\log(S) + dS\log(d)$. Moreover, the parameter count for the classic self-attention is $4d^2 + 4d$, while it is zero for 2DFFT. When the sequence length is 100 and the number of channels is 64, the classic self-attention requires 16 K parameters. However, when the number of channels increases to 512, the parameter count reaches 1 M. It is also worth mentioning that the above parameter counts are for a single transformation; the total count of transformation parameters in the overall model will increase linearly with the number of encoding blocks.

In summary, 2DFFT effectively emulates the classical self-attention mechanism in addition to offering enhanced computational efficiency. Unlike the substantial computational demands of the classical self-attention mechanism, the Fourier transform requires no predetermined parameters. Therefore, we opt to use 2DFFT for the implementation of the attention mechanism.

3.4. Feedforward Layer

This section outlines the experiments designed to specifically analyze optimization methods for the feedforward layer. Firstly, a comparison is made between the single-layer "Add + Norm" structure and the classical bilinear layer structure. The formula for the classical bilinear layer is given by:

$$\begin{cases} y' = \text{BatchNorm}(\text{SelfAttention}(x) + x) \\ \text{FFN}(y') = \text{GELU}(y'W_1 + b_1)W_2 + b_2 \\ y = \text{BatchNorm}(\text{FFN}(y') + y') \end{cases} \tag{5}$$

Here, x represents the original input data; y' is the output after applying self-attention to x; W_1, b_1, W_2, b_2 are the weights and biases of the feedforward neural network within the Transformer layer, respectively; and y is the final output of the layer.

From the experimental results in Tables 5 and 6, it can be observed that simplifying the feedforward layer not only did not compromise experimental performance but actually improved it. This is attributed to the fact that the feedforward layer does not perform any attention mechanism computations. In Transformer-like networks, the feedforward layer is responsible for introducing residual connections and enhancing the modeling capacity of the model through linear transformations. In the network model proposed in this paper, these tasks are handled by the convolutional branch, allowing for simplification of the feedforward layer, which is a module with complex computational requirements.

Table 5. Comparison of the accuracy of classic FFN and simplified FFN in classification tasks.

Dataset	Classic FFN	Simplified FFN
EthanolConcentration	0.2357	0.2586
FaceDetection	0.6720	0.6595
Handwriting	0.6824	0.6647
Heartbeat	0.6634	0.6829
JapaneseVowels	0.9108	0.9162
NATOPS	0.8500	0.8833
PEMS-SF	0.8555	0.8035
PhonemeSpectra	0.2219	0.2320
RacketSports	0.8487	0.8553
SpokenArabicDigits	0.9000	0.8835
StandWalkJump	0.4000	0.4667
UWaveGestureLibrary	0.8563	0.8844

Table 6. Comparison of the RMSE of classic FFN and simplified FFN in forecast tasks.

Dataset	Classic FFN	Simplified FFN
AppliancesEnergy	3.4574	3.0068
Covid3Month	0.1126	0.0809
FloodModeling1	0.0360	0.0240
FloodModeling2	0.0351	0.0209
FloodModeling3	0.0549	0.0413
IEEEPPG	35.6328	25.1032
LiveFuelMoistureContent	57.4598	51.3536

3.5. Global Frequency Pool

In this section, the experiments are designed to analyze the global frequency pooling layer with two variables: the frequency domain transformation method and the feature selection method. The experiments reveal that due to the diversity of time series tasks, it is challenging to select a single best-performing frequency pooling method. Different frequency domain transformation methods and feature extraction methods show significant differences in performance for different tasks. However, through experimenting with vari-

ous frequency pooling methods, it is found that introducing frequency domain information can greatly enhance experimental results, both for classification and prediction tasks.

Tables 7 and 8 present a comparison between the optimal models using different global frequency pooling methods and global average pooling. From the experimental data in the tables, it is evident that, in the seven prediction tasks listed, all frequency pooling methods outperform global average pooling. Among the twelve classification tasks, only one task exhibits lower classification accuracy with frequency pooling compared to global average pooling. Additionally, for datasets such as PEMS-SF and StandWalkJump, global frequency pooling significantly improves classification accuracy.

Table 7. Comparison of the accuracy of global average pool and global frequency pool in classification tasks.

Dataset	Global Average Pool	Global Frequency Pool
EthanolConcentration	0.2776	0.3004
FaceDetection	0.6839	0.6862
HandMovementDirection	0.5270	0.5270
Heartbeat	0.6537	0.7707
JapaneseVowels	0.8919	0.9162
NATOPS	0.8778	0.9000
PEMS-SF	0.7977	0.8786
PhonemeSpectra	0.2171	0.2574
RacketSports	0.8684	0.8947
SpokenArabicDigits	0.8481	0.8249
StandWalkJump	0.3333	0.6667
UWaveGestureLibrary	0.8625	0.9063

Table 8. Comparison of the RMSE of global frequency pool and global average pool in forecast tasks.

Dataset	Global Average Pool	Global Frequency Pool
AppliancesEnergy	2.8104	2.6043
Covid3Month	0.0575	0.0477
FloodModeling1	0.0539	0.0168
FloodModeling2	0.0294	0.0209
FloodModeling3	0.0323	0.0241
IEEEPPG	25.0910	23.3082
LiveFuelMoisture	50.4666	46.3127

4. Customized Computational Deployment

Deep learning networks are divided into two major steps: training and inference. During the training process, weights are continuously updated through learning, while in the inference process, the weights are fixed and no longer updated. The customized computational implementation in this work is focused on the inference part of the model. Leveraging the parallel nature of the dual-branch network and harnessing the flexible and efficient characteristics of FPGA devices, the algorithm undergoes customized computational optimization to accelerate the inference process.

4.1. Parallel Pipeline Design

This paper implements the entire system with customized computation deployment, and the overall network structure consists of three parts: the convolutional branch, attention branch, and pooling output. The convolutional branch consists of six identical convolution modules, each module containing six convolution layers computed in parallel. The convolutional branch and attention branch run in parallel. The attention branch is composed of three attention modules, each module containing two FFT and one "Add + Norm" operation. Through experimental design and optimization, the runtime of one attention module is approximately close to the computation time of two convolution modules. This

structure enables the convolutional branch and attention branch to complete computations almost simultaneously and input data into the global frequency domain pooling layer. The pipeline structure allows each submodule to compute different tasks simultaneously, improving the model throughput. The specific pipeline architecture is shown in Figure 1:

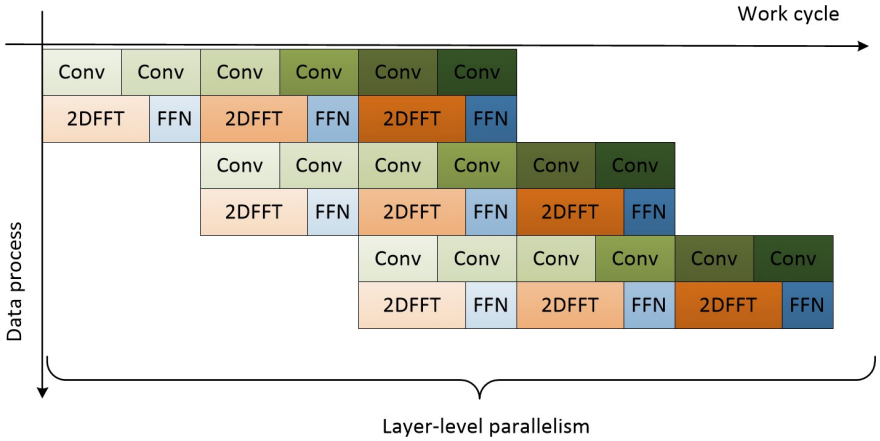


Figure 1. Dual-branch parallel pipeline structure, where the convolutional branch and the attention branch operate in parallel.

4.2. Operator Fusion

In the customized computation implementation, we have chosen DCT and low-dimensional information as features for global frequency domain pooling. We employ the matrix of DCT coefficients for the implementation of DCT. In the model design, the frequency domain pooling is followed by the fully connected layer used for output results. The DCT transformation and the fully connected layer can be expressed as follows:

$$\begin{cases} x_c = W_c x \\ y = W x_c + b \end{cases} \tag{6}$$

Here, x represents the input data, x_c denotes the output features of the DCT transformation combined with low-dimensional information extraction, y is the output data, representing either classification or prediction results, W_c is the DCT coefficient matrix with only the first P rows, W is the weight matrix of the fully connected layer, and b is the bias matrix. The frequency domain pooling and the fully connected layer can be fused, constructing a new computation method:

$$y = W_n x + b \tag{7}$$

where $W_n = W_c \cdot W$, and W_n can be precomputed and deployed in the FPGA system. This operator fusion greatly simplifies the pooling output layer, requiring only the parameters and computational cost of the final fully connected layer to complete the entire process of frequency domain transformation, feature extraction, and output computation.

4.3. System Structure

This paper employs the Xilinx Ultra96V2 (144 GOPS @200 MHz) device manufactured by Xilinx, San Jose, CA, USA for experimentation. The device is an edge computing device with limited on-chip resources, featuring 0.95 M on-chip memory and limited storage resources. The Ultra96V2 is a PYNQ development board that supports upper-level CPU programming in Python for task scheduling.

To enhance operational efficiency, this paper quantizes single-precision floating-point numbers into 8-bit fixed-point numbers. After quantization, the parameter count of the basic model (consisting of six convolutional modules and three attention modules) is

37.28 M. Due to the limited on-chip memory, it is not feasible to deploy all parameter information on-chip. Therefore, the parameters are deployed in external storage DRAM.

As shown in Figure 2, the upper-level CPU in the system is responsible for task scheduling and utilizes DMA to transfer data. Time series data are stored in DRAM, and two buffers are allocated, employing a ping-pong buffer structure for data preparation and preprocessing. This optimized data transfer structure aims to improve the throughput of the entire system.

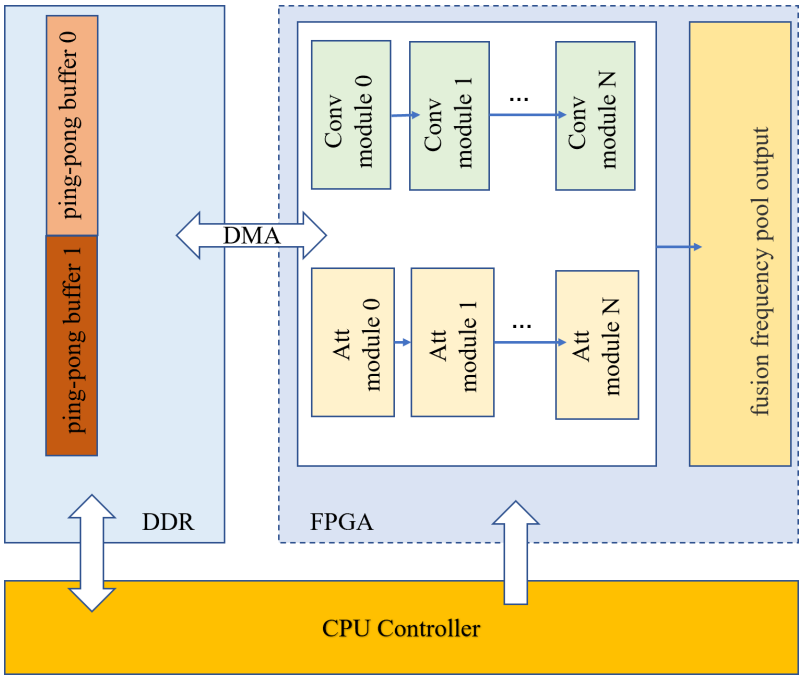


Figure 2. Custom computing architecture.

4.4. Results and Discussion

Here, a summary of the results and comparisons between the proposed model and other state-of-the-art time series algorithms, including InceptionTime, LSTM, BiLSTM, ResNet, and MiniRocket is presented. The tables present the optimal performance metrics for each algorithm, and an additional column indicates the ranking of the proposed model among all models in the experimental data.

From the experimental results presented in Tables 9 and 10, it is evident that the proposed algorithm in this paper demonstrates excellent performance in both classification and prediction tasks when compared to five other advanced time series algorithms. However, we observed that our model achieved the lowest RMSE on the AppEnergy dataset but ranked fourth in the Covid3Month dataset. This finding necessitated a deeper analysis of the performance variations. Firstly, the AppEnergy dataset is characterized by its series having more pronounced periodicity and regularity, which aligns well with our model's strong capability in trend capturing and periodic analysis. Our model effectively identifies and leverages these characteristics to enhance prediction accuracy. In contrast, the Covid3Month dataset's time series may be influenced by nonlinear factors and abrupt events, complicating the forecasting process. While our model excels in capturing long-term trends, it may not be as adept as some specially designed models in handling such nonlinear and sudden changes. In summary, the proposed model consistently achieves top-ranked results in most datasets, often securing the first position in terms of experimental effectiveness. While the model is not flawless and, in some tasks, may rank second or third, the differences are marginal compared to the optimal results for those tasks. Overall, the model presented in this paper attains state-of-the-art performance across a variety of time series tasks.

Table 9. The comparison of accuracy in classification tasks with other methods.

Dataset	InceptionTime	LSTM	BiLSTM	ResNet	MiniRocket	Ours	Ranking
EthanolConcen	0.2510	0.2471	0.2510	0.2510	0.2548	0.3004	1
FaceDetection	0.6617	0.5701	0.5843	0.5392	0.5831	0.6862	1
Handwriting	0.5953	0.0671	0.0565	0.4576	0.4671	0.5270	2
Heartbeat	0.6780	0.6439	0.6390	0.5171	0.6878	0.7707	1
JPVowels	0.9514	0.7811	0.7973	0.9216	0.5919	0.9162	3
NATOPS	0.9000	0.6889	0.6778	0.8833	0.8667	0.9000	1
PEMS_SF	0.8324	0.7977	0.8902	0.8266	0.8208	0.8786	2
RacketSports	0.8421	0.7039	0.7039	0.7961	0.7237	0.8947	1
SpokenArabic	0.5621	0.7854	0.8568	0.5939	0.3483	0.8249	2
StandWalkJump	0.4000	0.2000	0.3333	0.4000	0.4000	0.6667	1
UWaveGesture	0.6844	0.3500	0.3656	0.7063	0.5969	0.9063	1

Table 10. The comparison of RMSE in forecast tasks with other methods.

Dataset	InceptionTime	LSTM	BiLSTM	ResNet	MiniRocket	Ours	Ranking
AppEnergy	10.2361	7.7370	5.7399	9.9694	11.3129	2.6043	1
Covid3Month	0.0478	0.0431	0.0424	0.0464	1.2444	0.0477	4
FloodModeling1	0.0179	0.0181	0.0180	0.0169	1.0712	0.0168	1
FloodModeling2	0.0128	0.0183	0.0181	0.0141	0.8482	0.0129	2
FloodModeling3	0.0228	0.0224	0.0224	0.0225	0.6371	0.0214	1
IEEEPPG	31.6124	53.8133	31.089	42.7977	39.7816	23.3082	1
LiveFuelMois	47.4372	41.1614	41.1985	46.4096	42.2263	41.3127	3

In this section, a system test is conducted using three-channel time series data with a length of 512 as an example. The final operating frequency of the customized computing system is set at 200 MHz, with a power consumption of 6.380 W. The system achieves a speed of 32.6 FPS, and the resource utilization is summarized in Table 11. Compared to the 300 W power consumption of the Tesla V100, the customized computing system demonstrates a 47.02 times reduction in power consumption.

Table 11. Resource utilization.

Resource	Utilization	Available	Utilization (%)
LUT	49,620	70,560	70.32
FF	79,823	141,120	56.56
BRAM	200	216	92.59
DSP	360	360	100.0

5. Conclusions

This paper presents a customized computing algorithm design for multidimensional time series, introducing a dual-branch model with a parallel architecture. The use of positional encoding and global frequency domain pooling both enhance the model's performance in different ways, while the utilization of two-dimensional fast Fourier transform and simplified feedforward layers greatly reduces the number of parameters and improves operational efficiency. Comparative results with other advanced time series algorithms on public datasets demonstrate the superior performance of our algorithm. Additionally, the algorithm proposed in this paper is implemented in edge devices for custom computational deployment. The design includes a coarse-grained dual-branch parallel architecture and a fine-grained parallel structure for convolution operations. Furthermore, a pipeline architecture between submodules is designed to enhance overall operational efficiency, significantly reducing the overall system power consumption.

Author Contributions: Conceptualization, J.Y.; investigation, J.Y.; methodology, J.Y.; supervision, J.Y., Y.F., and Z.H.; validation, J.Y.; writing—original draft, J.Y.; writing—review and editing, J.Y. and Z.H. All authors have read and agreed to the published version of the manuscript.

Funding: This work is supported by the National Key Research and Development Project (Grant No. 2021YFB2206302) and the National Ministry of Industry and Information Technology High-quality Development Project (Grant No. E412151).

Data Availability Statement: The data presented in this study are available in this article.

Acknowledgments: The authors would like to express their gratitude to all those who helped them during the writing of this manuscript. The authors owe their sincere gratitude to friends and colleagues who gave them much enlightening advice and encouragement.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A.N.; Kaiser, Ł.; Polosukhin, I. Attention is all you need. *Adv. Neural Inf. Process. Syst.* **2017**, *30*, 5998–6008.
2. Devlin, J.; Chang, M.W.; Lee, K.; Toutanova, K. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv* **2018**, arXiv:1810.04805.
3. Radford, A.; Narasimhan, K.; Salimans, T.; Sutskever, I. Improving Language Understanding by Generative Pre-Training. 2018. Preprint. Available online: https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf (accessed on 28 February 2024).
4. Zerveas, G.; Jayaraman, S.; Patel, D.; Bhamidipaty, A.; Eickhoff, C. A transformer-based framework for multivariate time series representation learning. In Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining, Virtual, 14–18 August 2021; pp. 2114–2124.
5. Ismail Fawaz, H.; Lucas, B.; Forestier, G.; Pelletier, C.; Schmidt, D.F.; Weber, J.; Webb, G.I.; Idoumghar, L.; Muller, P.A.; Petitjean, F. Inceptiontime: Finding alexnet for time series classification. *Data Min. Knowl. Discov.* **2020**, *34*, 1936–1962.
6. Lin, Y.; Koprinska, I.; Rana, M. Temporal convolutional attention neural networks for time series forecasting. In Proceedings of the 2021 International Joint Conference on Neural Networks (IJCNN), Shenzhen, China, 18–22 July 2021; pp. 1–8.
7. Shih, S.Y.; Sun, F.K.; Lee, H.Y. Temporal pattern attention for multivariate time series forecasting. *Mach. Learn.* **2019**, *108*, 1421–1441.
8. Qin, Y.; Song, D.; Chen, H.; Cheng, W.; Jiang, G.; Cottrell, G. A dual-stage attention-based recurrent neural network for time series prediction. *arXiv* **2017**, arXiv:1704.02971.
9. Khodabakhsh, A.; Ari, I.; Bakır, M.; Alagoz, S.M. Forecasting multivariate time-series data using LSTM and mini-batches. In *Data Science: From Research to Application*; Springer: Cham, Switzerland, 2020; pp. 121–129.
10. Guo, M.H.; Liu, Z.N.; Mu, T.J.; Hu, S.M. Beyond self-attention: External attention using two linear layers for visual tasks. *IEEE Trans. Pattern Anal. Mach. Intell.* **2022**, *45*, 5436–5447.
11. Wu, C.; Wu, F.; Qi, T.; Huang, Y.; Xie, X. Fastformer: Additive attention can be all you need. *arXiv* **2021**, arXiv:2108.09084.
12. Xu, J.; Sun, X.; Zhang, Z.; Zhao, G.; Lin, J. Understanding and improving layer normalization. *Adv. Neural Inf. Process. Syst.* **2019**, *32*, 4383–4393.
13. Shen, S.; Yao, Z.; Gholami, A.; Mahoney, M.; Keutzer, K. Powernorm: Rethinking batch normalization in transformers. In Proceedings of the International Conference on Machine Learning, Virtual, 12–18 July 2020; pp. 8741–8751.
14. Shazeer, N. Glu variants improve transformer. *arXiv* **2020**, arXiv:2002.05202.
15. Shazeer, N.; Mirhoseini, A.; Maziarz, K.; Davis, A.; Le, Q.; Hinton, G.; Dean, J. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv* **2017**, arXiv:1701.06538.
16. Wu, Z.; Liu, Z.; Lin, J.; Lin, Y.; Han, S. Lite transformer with long-short range attention. *arXiv* **2020**, arXiv:2004.11886.
17. Sukhbaatar, S.; Grave, E.; Bojanowski, P.; Joulin, A. Adaptive attention span in transformers. *arXiv* **2019**, arXiv:1905.07799.
18. Yang, Y.; Wang, L.; Shi, S.; Tadepalli, P.; Lee, S.; Tu, Z. On the sub-layer functionalities of transformer decoder. *arXiv* **2020**, arXiv:2010.02648.
19. Lin, M.; Chen, Q.; Yan, S. Network in network. *arXiv* **2013**, arXiv:1312.4400.
20. Qin, Z.; Zhang, P.; Wu, F.; Li, X. Fcanet: Frequency channel attention networks. In Proceedings of the IEEE/CVF international Conference on Computer Vision, Virtual, 11–17 October 2021; pp. 783–792.
21. Lee-Thorp, J.; Ainslie, J.; Eckstein, I.; Ontanon, S. Fnet: Mixing tokens with fourier transforms. *arXiv* **2021**, arXiv:2105.03824.
22. Tan, C.W.; Bergmeir, C.; Petitjean, F.; Webb, G.I. Monash University, UEA, UCR time series extrinsic regression archive. *arXiv* **2020**, arXiv:2006.10996.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.